Kampüs Kaynak Rezervasyon Sistemi: Kural Tabanlı Doğrulama, Doluluğa Duyarlı Fiyatlandırma ve Olay Güdümlü Simülasyon

Hasan Huseyin YUKSEL
240201130
Kocaeli Universitesi
yukselhhuseyin58@gmail.com

HANI ASHRAFI
240201152
Kocaeli Universitesi
haofficial122@gmail.com

***Özet*—Bu çalışmada, sınırlı sayıdaki toplantı odası, laboratuvar, üç boyutlu yazıcı ve GPU sunucusunun öğrenci, araştırmacı ve teknisyenler arasında paylaşımını düzenleyen bir rezervasyon modülü ile bu modülü doğrudan kullanan bir masaüstü uygulaması geliştirilmiştir. Modül; 13 adımlı sıralı doğrulama, 15 dakikalık dilimler ve iki saatlik bantlar üzerinden doluluğa duyarlı fiyatlandırma, "ya hepsi ya hiçbiri" paket rezervasyonu, beş durumlu bir durum makinesi, güvenilirliğe göre öncelikli bekleme listesi ve olay güdümlü simüle saat ilerletmeyi uygular. Uygulama yalnızca Python standart kütüphanesiyle yazılmıştır; kesirli aritmetik sayesinde yuvarlama kuralı kayan nokta hatasından etkilenmez. Tkinter ile geliştirilen arayüz iş kuralı içermez ve Türkçe, İngilizce ile sağdan sola Farsça olmak üzere üç dili destekler. Örnek senaryo beklenen çıktıyla bayt düzeyinde aynı sonucu vermiş, elle hesaplanan 119 beklenen satırın tamamı geçmiş ve bağımsız bir gerçekleştirimle 2536 rastgele komut üzerinde yapılan farksal testte hiçbir fark bulunmamıştır.**

***Anahtar Kelimeler*—kaynak rezervasyonu, durum makinesi, dinamik fiyatlandırma, ayrık olay simülasyonu, kullanıcı arayüzü tasarımı.**

I. GIRIS

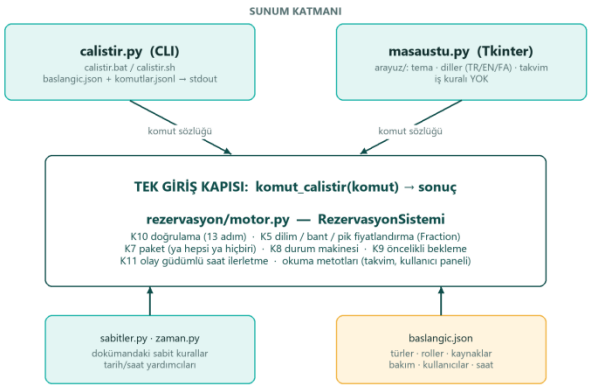
Üniversitelerde pahalı ve sınırlı ekipmanın ortak kullanımı; rezervasyon alıp gelmeme, cihazların iki kullanım arasında soğuma veya havalandırma ihtiyacı ve yoğun saatlerde talebin yığılması gibi sorunlar doğurur. Proje dokümanı bu sorunları Kaynak Puanı (KP) ekonomisi, türe göre değişen hazırlık süreleri ve güvenilirlik puanıyla çözen K1–K11 kurallarını tanımlar.

Bu çalışmanın amaçları şunlardır: (i) kuralları arayüzden bağımsız, dosya tabanlı bir modülde birebir uygulamak; (ii) modülü tek bir giriş noktasından kullanan ve iş kuralı içermeyen bir masaüstü uygulaması geliştirmek; (iii) doğruluğu bağımsız testlerle göstermek. Gizli testlerin farklı parametrelerle hazırlanacağı belirtildiği için başlangıç dosyasındaki hiçbir değer koda gömülmemiştir.

II. YONTEM

A. Mimari

Sistem üç katmandan oluşur (Şekil 1). Alt katmanda dokümandaki sabit kurallar (`sabitler.py`: 15 dk adım, 08:00–22:00 çalışma saati, 60 dk art arda kullanım boşluğu vb.) ve zaman yardımcıları (`zaman.py`) bulunur. Orta katmandaki `motor.py`, sistemin tüm durumunu tek bir `RezervasyonSistemi` nesnesinde tutar. Komut satırı programı (`calistir.py`) ve masaüstü uygulaması (`masaustu.py`) motora yalnızca `komut_calistir(komut)` metodu üzerinden erişir. Bu tek giriş kapısı iki arayüzün aynı davranmasını garanti eder; her komut için `no`, `komut` ve `durum` alanlarını içeren bir sonuç üretilir ve oturum günlüğüne eklenir.



Şekil 1. Katmanlı mimari ve tek giriş kapısı

B. Veri Modeli

Başlangıç dosyasındaki türler, roller, kaynaklar, bakım pencereleri ve kullanıcılar JSON yapısıyla birebir örtüşen sözlüklerde tutulur. Kaynak, kullanıcı, rezervasyon ve bekleme kayıtları kimliğe göre sözlüklere yerleştirilir; böylece erişim sabit zamanda yapılır. Python sözlükleri ekleme sırasını koruduğu için rezervasyonlar numara sırasıyla gezilir ve K11'deki "numara küçükten büyüğe" kuralı ek sıralama gerektirmeden sağlanır (Tablo I).

TABLO I
TEMEL VERI YAPILARI

Varlık	Alanlar
Rezervasyon	id, no, kullanici, kaynak, baslangic, bitis, ucret, durum, paket
Bekleme kaydı	id, no, kullanici, kaynak, baslangic, bitis, kayit_saati
Kullanıcı	id, ad, rol, sertifikalar, bakiye, guvenilirlik
Kaynak türü	hazirlik_dk, temel_ucret, enerji_yogun, sertifika
Rol	haftalik_kota, ileri_gun, azami_sure_dk

C. Kurallarin Uygulanisi

1) *Doğrulama (K1–K4, K10):* `dogrula()` metodu 13 adımı dokümandaki sırayla uygular ve ilk başarısız adımın kodunu döndürür. İki bayrak aynı metodun farklı komutlarda yeniden kullanılmasını sağlar: `bakiye_bak=False` (fiyat, paket, bekleme) ve `cakisma_bak=False` (bekleme). Aralıklar yarı

açıktır; kesişme $b < e' \wedge e > b'$, hazırlık ihlali ise (1) ile sınanır. Art arda kullanımda h yerine 60 dk alınır. TAMAMLANDI durumu hazırlıkta sayılırken GELMEDI ve IPTAL hiçbir kontrolde sayılmaz. Tablo II doğrulama sırasını özetler; örneğin dokümandaki 6. komutta hem hazırlık hem art arda kullanım ihlal edildiği hâlde sıra gereği HAZIRLIK_IHLALI döner.

$$b < e' + h \quad \wedge \quad e + h > b'$$

TABLO II
REZERVASYON DOĞRULAMA SIRASI (K10)

#	Kod	Koşul
1	BULUNAMADI	Kullanıcı veya kaynak yok
2	ZAMAN_ADIMI_HATASI	15 dk adımı yok veya e ≤ b
3	GECMIS_ZAMAN	b < simüle saat
4	CALISMA_SAATI_DISI	08:00–22:00 dışı veya gün aşımı
5	SURE_SINIRI	< 30 dk veya > azami_sure_dk
6	ILERI_TARIH_SINIRI	b > saat + ileri_gun (güv. < 50 ise 1 gün)
7	ROL_IZNI_YOK	Rol izinli_roller içinde değil
8	SERTIFIKA_YOK	Türün sertifikası kullanıcıda yok
9	BAKIM	Bakım penceresiyle kesişme
10	KESISME	ONAYLI/KULLANIMDA ile kesişme
11	HAZIRLIK_IHLALI	Denklem (1), TAMAMLANDI dahil
12	ARDISIK_KULLANIM	Aynı kişi/kaynak, boşluk < 60 dk
13	BAKIYE_YETERSIZ	ücret > bakiye

Algoritma 1. 13 adimli sirali dogrulama

```
dogrula(u_id, k_id, b, e, bakiye_bak=True,
cakisma_bak=True):
u ← kullanicilar.get(u_id) ; k ← kaynaklar.get(k_id)
eğer u = None ∨ k = None ise döndür BULUNAMADI
eğer b.dk mod 15 ≠ 0 ∨ e.dk mod 15 ≠ 0 ∨ e ≤ b ise
döndür ZAMAN_ADIMI_HATASI
eğer b < saat ise döndür GECMIS_ZAMAN
eğer b < b(08:00) ∨ e > b(22:00) ise döndür
CALISMA_SAATI_DISI
eğer e − b < 30 dk ∨ e − b > rol.azami_sure_dk ise
döndür SURE_SINIRI
ileri ← (1 gün eğer u.guvenilirlik < 50 aksi hâlde
rol.ileri_gun)
eğer b > saat + ileri ise döndür ILERI_TARIH_SINIRI
eğer u.rol ∉ k.izinli_roller ise döndür ROL_IZNI_YOK
eğer tur.sertifika ∉ u.sertifikalar ise döndür
SERTIFIKA_YOK
eğer ∃m ∈ bakimlar(k_id) : b < m.bitis ∧ e >
m.baslangic ise döndür BAKIM
eğer cakisma_bak ise:
eğer ∃r ∈ aktif(k_id) : b < r.bitis ∧ e > r.baslangic
ise döndür KESISME
eğer ∃r ∈ hazirlik_sayilan(k_id) : b < r.bitis+h ∧
e+h > r.baslangic ise
döndür HAZIRLIK_IHLALI
eğer ∃r ∈ kullanici_rez(u_id, k_id) : b < r.bitis+60 ∧
e+60 > r.baslangic ise
döndür ARDISIK_KULLANIM
eğer bakiye_bak ∧ ucret_hesapla(k_id, b, e) > u.bakiye
ise döndür BAKIYE_YETERSIZ
döndür GECERLI
```

2) Fiyatlandırma (K5): Talep 15 dakikalık dilimlere bölünür. Her dilim için düştüğü iki saatlik bandın doluluğu, aynı türdeki ONAYLI ve KULLANIMDA rezervasyonların bant içindeki dakikalarının kapasiteye (kaynak sayısı × 120) oranıdır. Bant çarpanı kayan nokta kullanılmadan, tam sayı bölmesiyle hesaplanır:

$$k = \lfloor\, 4 \cdot \text{dolu} / \text{kapasite} \rfloor\,, \quad c_bant = 1 + k / 4$$

$$\text{ücret} = \lfloor\, \Sigma\, t \cdot ¼ \cdot c_bant \cdot c_pik \;+\; ½ \,\rfloor$$

Burada t saatlik temel ücret, *c_pik* enerji yoğun türlerde 10:00–16:00 arasında 1,5, diğer durumlarda 1'dir. Python'un `round()` fonksiyonu yarımları çift sayıya yuvarladığı için (round(22,5) = 22) kullanılmamış; toplam `fractions.Fraction` ile kesin olarak tutulmuştur.

Örnek: Dokümandaki 2. komutta (u2, YAZ-2, 10:00–12:00) 10–12 bandında r1'in 60 dakikası vardır. İki yazıcı olduğundan doluluk 60/240 = 0,25, $k = \lfloor 4 \cdot 60/240 \rfloor = 1$ ve *c_bant* = 1,25'tir. Sekiz dilimin hepsi pik saattedir: 8 × (12 × ¼) × 1,25 × 1,5 = 45 KP. Kendi senaryomuzda 22,5 KP'lik toplam 23'e yuvarlanmış; `round()` kullanılsaydı 22 çıkacaktı. Hesap Algoritma 2'de özetlenmiştir.

Algoritma 2. Ücret hesabı (K5)

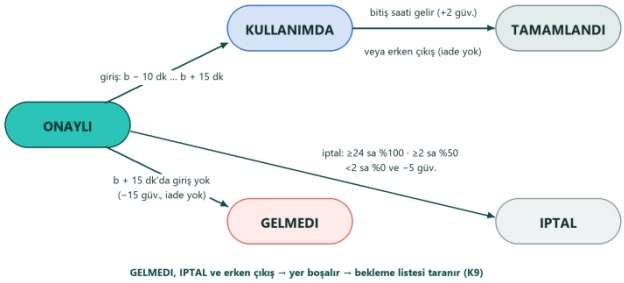
```
ucret_hesapla(kaynak_id, b, e):
kapasite ← |aynı türdeki kaynaklar| × 120
dilim_temel ← Fraction(tur.temel_ucret) × Fraction(15,
60)
toplam ← Fraction(0) ; an ← b
an < e oldukça:
bant ← an'ın 2 saatlik bandı [bant_bas, bant_bit)
dolu ← Σ ortak_dk(ONAYLI/KULLANIMDA aynı tür, bant)
kademe ← min(4, (4 × dolu) // kapasite)
c_bant ← 1 + Fraction(kademe, 4)
c_pik ← 1.5 (enerji yoğun ∧ 10 ≤ an.saat < 16) aksi
hâlde 1
toplam ← toplam + dilim_temel × c_bant × c_pik
an ← an + 15 dk
döndür ⌊toplam + 1/2⌋
```

3) *Paket (K7):* Önce kullanıcı, kalem sayısı ve güvenilirlik denetlenir. Ardından tüm kalemler paketten önceki duruma göre, bakiye hariç doğrulanır ve fiyatlanır; diğer kalemler doluluğa katılmaz. Kalemler arası kesişme, hazırlık ve 60 dk kontrolü ikili döngüyle yapılır. Rezervasyonlar ancak bütün denetimler geçerse tek seferde oluşturulur (atomiklik).

Algoritma 3. Atomik paket rezervasyonu (K7)

```
komut_paket(u_id, kalemler):
eğer u_id ∉ kullanicilar ise döndür RED(BULUNAMADI)
eğer |kalemler| < 2 ∨ |kalemler| > 4 ise döndür
RED(PAKET_BOYUTU)
eğer u.guvenilirlik < 50 ise döndür
RED(GUVENILIRLIK_DUSUK)
her k ∈ kalemler için:
(k.hata, k.ucret) ← dogrula(u_id, k.kaynak, k.b, k.e,
bakiye_bak=False)
eğer ∃k : k.hata ≠ GECERLI ise döndür RED(PAKET_RED)
her (a, b) ∈ kalemler × kalemler (a ≠ b ∧ a.kaynak =
b.kaynak) için:
eğer kesisir(a, b, max(h, 60 dk)) ise döndür
RED(PAKET_ICI_KESISME)
toplam ← Σ k.ucret
eğer toplam > u.bakiye ise döndür RED(BAKIYE_YETERSIZ)
her k ∈ kalemler için rezervasyon_ekle(u_id, k.kaynak,
k.b, k.e, k.ucret)
döndür TAMAM
```

4) Durum makinesi ve iade (K6, K8): Şekil 2'deki geçişler uygulanır. İade başlangıca kalan süreye göre %100, %50 (aşağı yuvarlanır) veya %0'dır; `iade_ekle()` bakiyenin haftalık kotayı aşmasını engeller ve bakiyeye gerçekten eklenen miktarı döndürür.



Şekil 2. Rezervasyon durum makinesi.

5) Bekleme listesi (K9): Kayıtlar (−güvenilirlik, kayıt saati, numara) demetine göre sıralanır. Yer boşaldığında (iptal, gelmeme, çıkış) liste baştan sona taranır; o anki saat ve bakiyeyle 13 kontrolün tamamını geçen kayıt otomatik rezervasyona dönüşür, geçemeyen listede kalır. Bekleme komutu iki geçişli doğrulama kullanır: önce kesişme ve hazırlık dışındaki kontroller, sonra tam kontrol. Böylece talebin *yalnızca* başka bir rezervasyon nedeniyle reddedildiği durum ayırt edilir.

Algoritma 4. Bekleme kaydi ve oncelikli tarama

```
komut_bekleme(u_id, k_id, b, e):
eğer dogrula(u_id, k_id, b, e, bakiye_bak=False,
cakisma_bak=False) ≠ GECERLI ise
döndür RED(hata)
eğer dogrula(u_id, k_id, b, e, bakiye_bak=False,
cakisma_bak=True) = GECERLI ise
döndür RED(BEKLEME_GEREKSIZ)
beklemeler.ekle({u_id, k_id, b, e, kayit_saati: saat})
döndür TAMAM
bekleme_tara(k_id):
baslangic < saat olan eski beklemeleri sil
adaylar ← k_id beklemelerini (-guvenilirlik,
kayit_saati, no) ile sırala
her w ∈ adaylar için:
(hata, ucret) ← dogrula(w.u_id, k_id, w.b, w.e)
eğer hata = GECERLI ise:
rezervasyon_ekle(w.u_id, k_id, w.b, w.e, ucret)
beklemeler.sil(w)
```

6) Saat ilerletme (K11): Dakika dakika simülasyon yerine ayrık olay simülasyonundaki sonraki olay zamanı yaklaşımı kullanılmıştır . Aday anlar (sonraki Pazartesi 00:00, KULLANIMDA bitişleri, ONAYLI başlangıç + 15 dk) arasından (eski, yeni] aralığındaki en erkeni seçilir, saat o ana alınır ve aynı andaki olaylar dokümandaki öncelikle işlenir (Algoritma 4). Her turda saat kesin olarak ilerlediği için döngü sonludur.

Algoritma 5. Olay güdümlü saat ilerletme ve bekleme tarama

```
saati_ilerlet(hedef):
olaylar ← []
döngü:
an ← min{ t ∈ adaylar | saat < t ≤ hedef }
eğer an yoksa çık
saat ← an
Pazartesi 00:00 ise her u için u.bakiye ←
haftalik_kota
KULLANIMDA ∧ bitis = an → TAMAMLANDI, güv ← min(100,
güv + 2)
ONAYLI ∧ baslangic+15 = an → GELMEDI, güv ← max(0, güv
- 15)
boşalan her kaynak k için ( (-güv, kayit_saati, no)
sırasıyla ):
dogrula() geçen her bekleme kaydını ONAYLI
rezervasyona çevir
saat ← hedef ; baslangic < saat olan eski beklemeleri
sil
döndür olaylar
```

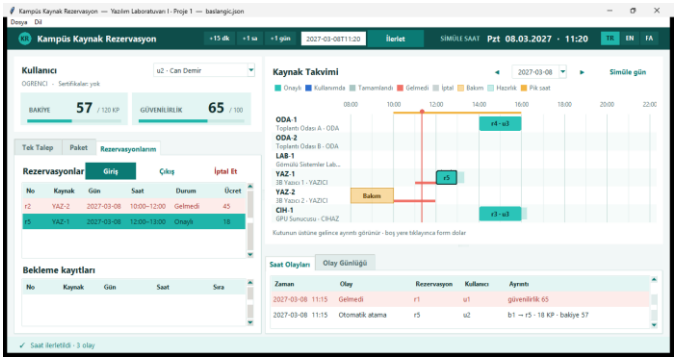
D. Masaüstü Uygulaması ve Arayüz Tasarımı

Uygulama, dokümanın istediği altı bölümü içerir: kaynak takvimi, tek ve paket talep formu, kullanıcı paneli, giriş/çıkış/iptal/bekleme eylemleri, simüle saat denetimi ve olay günlüğü (Şekil 3). Arayüz yalnızca komut sözlüğü üretip sonucu gösterir; ücret önizlemesi `fiyat` komutuyla, dilim ayrıntısı motorun salt okunur metoduyla alınır. Bu nedenle başlangıç dosyasında bir parametre değiştiğinde uygulama kod değişmeden yeni kurala uyar.

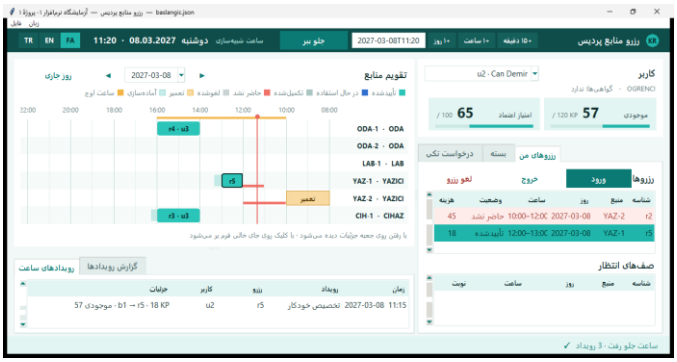
Görsel tasarımda firuze ana renk, renk çemberindeki tamamlayıcısı mercan ile çift tamamlayıcı bir uyum içinde kullanılmıştır . Tüm metin/zemin çiftleri WCAG 2.1 kontrast ölçütüne (≥ 4,5:1) göre ölçülmüştür. Boşluklar Fibonacci dizisinden (3–5–8–13–21–34) seçilmiş, ana bölmeler altın orana (0,382 / 0,618) göre yerleştirilmiştir . Ana eylem düğmeleri daha belirgindir (Fitts yasası); seçenekler sekmelere gruplanarak bir anda görünen karar sayısı azaltılmıştır (Hick yasası); geri alınamaz iptal işlemi onay ister (hata önleme ilkesi). Farsça seçildiğinde arayüz bütünüyle aynalanır, zaman ekseni sağdan sola akar ve karışık metinler Unicode RLE/PDF denetim karakterleriyle sarılarak doğru sırada gösterilir. Takvim satır yüksekliğini kaynak sayısına göre ayarladığından kaynak sayısı artsa da kaydırma gerekmeden sığar. Kullanılan renkler ve ölçülen kontrast oranları Tablo III'te verilmiştir; durumlar yalnızca renkle değil, kutu içindeki yazı ve şekille de ayrıldığından renk körlüğü olan kullanıcılar için de ayırt edilebilir.

TABLO III
RENK PALETI VE WCAG KONTRAST ORANLARI

Kullanım	Renk	Kontrast
Ana metin / zemin	#12302F / #F2F7F7	13,2 : 1
Ana düğme (beyaz yazı)	#0B7A74 firuze	5,2 : 1
ONAYLI kutusu	#2BC4B8 firuze	6,5 : 1
KULLANIMDA kutusu	#2F6FD0 gök mavisi	4,9 : 1
GELMEDI kutusu / hata	#F07167 mercan	5,9 : 1
Bakım (taralı)	#F2B544 kehribar	7,9 : 1
İkincil yazı / beyaz	#557071	5,3 : 1



Şekil 3. Masaüstü uygulaması (örnek senaryo, simüle saat 11:20)



Şekil 4. Aynı ekranın Farsça (sağdan sola, aynalanmış) görünümü.

III. DENEYSEL SONUCLAR

Doğruluk, dokümandaki örnek senaryo ve altı kendi senaryomuzla sınanmıştır (Tablo IV). Beklenen sonuçlar programdan kopyalanmamış, kurallar üzerinden elle hesaplanmıştır. Karşılaştırma dokümandaki ölçüte göre yapılmıştır: fazladan alan hata sayılmaz, eksik veya farklı alan hata sayılır.

TABLO IV
TEST SENARYOLARI VE SONUÇLARI

Senaryo	Odak	Sonuç
ornek	Doküman örneği	15/15
01	13 hata kodu, bant çarpanı, ½ yuvarlama	21/21
02	Giriş penceresi, erken çıkış, iade, bekleme	26/26
03	Paket hata sırası, atomiklik	12/12
04	Düşük güvenilirlik, haftalık sıfırlama, kota tavanı	22/22
05	Değiştirilmiş parametreler, yeni tür	15/15
06	Aynı andaki olayların sırası	8/8
Toplam		119/119

Örnek senaryoda `calistir.bat` çıktısı beklenen dosyayla bayt düzeyinde aynıdır. Senaryo 05'te hazırlık süresi, ücretler, kota, ileri tarih sınırı ve kaynak sayısı değiştirilmiş, yeni bir STUDYO türü eklenmiştir; aynı kod yeni kurallara uyum sağlamıştır (ör. üç yazıcıda 60 dk doluluk 60/360 → çarpan 1,00). Ayrıca 400 rastgele komut dizisi (toplam 17.571 komut) bağımsız olarak yazılmış ikinci bir gerçekleştirimle karşılaştırılmış; hiçbir satırda fark ve hiçbir çökme görülmemiştir. Ortalama işlem süresi komut başına yaklaşık 0,1 ms'dir (≈ 10.400 komut/s, Python 3.14). Masaüstü uygulamasında örnek komut dosyası çalıştırılıp beklenen dosyayla karşılaştırıldığında da 15/15 eşleşme elde edilmiştir.

IV. TASARIM KARARLARI

Kesirli aritmetik: "Tam 0,5 ise yukarı" kuralı kayan nokta hatasına duyarlıdır; `Fraction` sonucu her parametrede kesin kılar. *Sözlük tabanlı veri:* JSON ile birebir eşleşir ve kodun sözlü sınavda satır satır açıklanmasını kolaylaştırır. *Tek giriş kapısı:* Komut satırı ile arayüz aynı sonucu üretir ve oturum günlüğü doğrudan `sonuclar.jsonl` olarak kaydedilebilir. *Parametrelerin ayrılması:* Değişken değerler yalnızca başlangıç dosyasından okunur, dokümandaki sabit kurallar tek bir dosyada toplanmıştır. *Olay güdümlü saat:* Dakika adımlı simülasyona göre hızlıdır ve aynı andaki olayların sırasını açıkça belirler. *Sıfır bağımlılık:* Kurulum, ağ erişimi veya API anahtarı gerektirmez. *Tanımsız uç durum:* Başlangıçtan önce yapılan erken çıkışta bitiş başlangıca sabitlenir; böylece ters aralık oluşmaz.

V. SONUC

Geliştirilen modül K1–K11 kurallarının tamamını uygulamakta, örnek senaryoyu birebir geçmekte ve parametre değişikliklerine kod değişikliği olmadan uyum sağlamaktadır. Masaüstü uygulaması aynı motoru kullanan, üç dilli ve erişilebilirlik ölçütlerine dayalı bir arayüz sunmaktadır. Gelecekte oturum durumunun kalıcı olarak saklanması ve çok kullanıcılı ağ sürümü planlanmaktadır.

KAYNAKCA.

[1] Dynamic pricing from Wikipedia [Online]. Available: https://en.wikipedia.org/wiki/Dynamic_pricing

[2] Codes from W3schhool [Online]. Available: https://www.w3schools.com/python/

[3] Lessons Document from Kocaeli Uni [Online]. Available: http://edestek.kocaeli.edu.tr/

[4] Gemini [Online]. Available: https://gemini.google.com/app?hl=tr

[5] Claude[Online]. Available: https://claude.ai/new